

Demian RAG Pipeline

LangChain · Chroma · Hybrid Retrieval · RAGAS 평가

과제	GENAI_DAY4_RAG_프로젝트 — 6. 완성 예제
제출	추병곤
원 강의자료	박광석 (모두의연구소)
구성	Step 1 Loader → Step 2 Splitter → Step 3 Embedding → Step 4 Retriever → Step 5 QA → 정량 평가

```
Demian.pdf (스캔 OCR)
  | PyPDFLoader.load() + clean()           [A] 분절 복원 · 러닝헤더 제거
  ▼
RecursiveCharacterTextSplitter             [B] 500 tok / overlap 80 / tiktoken
  ▼
OpenAIEmbeddings (text-embedding-3-small)  [C] 1536d
  ▼
Chroma (persist, HNSW)                    [D]
  ▼
Hybrid Retriever = BM25(0.4) + MMR(0.6)    [1→3] k=4, fetch_k=20, λ=0.5
  ▼
Grounding Prompt (context-only · 거절 · 인용) [4]
  ▼
gpt-4o (T=0) → 답변 + 출처                [5]
  ▼
RAGAS: precision / recall / faithfulness / relevancy
```

last modified date : 2026.03.15

제작 : 박광석 (모두의연구소)

랭체인으로 RAG 시작하기

해당 노트는 Langchain으로 RAG를 구현하기 위해 필요한 각 컴포넌트인 Document Loaders, Text splitters, Text embeddings, Vectorstores, Retriever를 다룹니다

Step 0 : 설치와 준비

Langchain 설치 및 Gemini API 키를 등록하도록 합니다.

In

```
from google.colab import drive
drive.mount('/content/drive')
```

실행 결과

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

In

```
!pip install -U -q langchain langchain-openai
!pip install -U -q langchain-community langchain-core
!pip install -U langchain-text-splitters
```

실행 결과

(패키지 설치 로그 생략)

In

```
import os
```

In

```
from google.colab import userdata
os.environ['OPENAI_API_KEY'] = userdata.get('OPENAI_API_KEY')
```

In

```
#! curl ipinfo.io
```

실행 결과

(패키지 설치 로그 생략)

In

```
from langchain_openai import ChatOpenAI

# OpenAI API를 사용하는 설정으로 변경
# 모델명은 필요에 따라 "gpt-4o", "gpt-4-turbo", "gpt-3.5-turbo" 등으로 바꿀 수 있습니다.
llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.0,
)
```

In

```
!pip install -q pypdf pdf2image docx2txt pdfminer unstructured #의존성 모듈을 설치합니다
```

실행 결과

(패키지 설치 로그 생략)

Step 1 : Document Loaders 사용해보기

Document Loader는 다양한 형태의 원본 데이터를

LLM이 이해할 수 있는 Document 객체(text + metadata) 로 변환하는 역할을 합니다.

PDF, 웹페이지, CSV와 같이 형식이 서로 다른 문서들을 일관된 구조로 파싱하여, 이후 Chunking·Embedding·검색(Retrieval) 단계에서 바로 사용할 수 있도록 만들어줍니다.

즉, Document Loader는 **RAG 파이프라인의 가장 첫 단계에서 “데이터를 읽을 수 있는 형태로 정리하는 역할을 담당합니다.**

공식 문서에서는 지원되는 다양한 Loader 목록을 확인할 수 있습니다.

https://python.langchain.com/docs/modules/data_connection/document_loaders/

PDFLoader 사용

이번 실습에서는 가장 많이 사용되는 문서 형식인 PDF 파일을 대상으로 PyPDFLoader를 사용해 문서를 불러옵니다.

실습을 위해, 질의응답에 활용하고 싶은 PDF 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

PDFLoader는 각 페이지를 하나의 Document 단위로 변환하며, 이 단계에서 생성된 문서들은 이후 Text Splitter를 통해 의미 단위로 다시 분할됩니다.

In

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("/content/Demian.pdf")
pages = loader.load_and_split()
```

In

```
pages[0]
```

실행 결과

```
Document(metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture Plug-in', 'creator': 'ScanFix(TM) Enhanced', 'creationdate': '2015-09-10T01:40:29+00:00', 'moddate': '2019-01-30T17:47:47+01:00', 'source': '/content/Demian.pdf', 'total_pages': 182, 'page': 0, 'page_label': '1'}, page_content='DEMIAN \n• \nDownloaded from https://www.holybooks.com')
```

In

```
print(pages[10])
```

실행 결과

```
page_content='TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
... (출력 일부 생략)
```

출력 결과를 보기 쉽게 확인하기 위해, Document 객체 전체가 아닌 실제 텍스트 본문이 담긴 `page_content`만 선택하여 확인해 보겠습니다.

In

```
print(pages[10].page_content)
```

실행 결과

```
TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
... (출력 일부 생략)
```

CSVLoader

SV 파일은 행(row) 단위로 구조화된 데이터를 담고 있는 형식으로, LangChain의 CSVLoader를 사용하면 각 행을 하나의 Document 객체로 변환할 수 있습니다.

이렇게 변환된 문서들은 이후 PDF나 웹 문서와 동일하게 Embedding, VectorStore, Retrieval 단계에서 함께 활용할 수 있습니다.

실습을 위해, CSV 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

```
In
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("/content/titanic.csv")

data = loader.load()
```

```
In
data[:3]
```

실행 결과

```
[Document(metadata={'source': '/content/titanic.csv', 'row': 0}, page_content='PassengerId: 1\nSurvived: 0\nPclass: 3\nName: Braund, Mr. Owen Harris\nSex: male\nAge: 22\nSibSp: 1\nParch: 0\nTicket: A/5 21171\nFare: 7.25\nCabin: \nEmbarked: S'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 1}, page_content='PassengerId: 2\nSurvived: 1\nPclass: 1\nName: Cumings, Mrs. John Bradley (Florence Briggs Thayer)\nSex: female\nAge: 38\nSibSp: 1\nParch: 0\nTicket: PC 17599\nFare: 71.2833\nCabin: C85\nEmbarked: C'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 2}, page_content='PassengerId: 3\nSurvived: 1\nPclass: 3\nName: Heikkinen, Miss. Laina\nSex: female\nAge: 26\nSibSp: 0\nParch: 0\nTicket: STON/O2. 3101282\nFare: 7.925\nCabin: \nEmbarked: S')]
```

웹베이스로더

웹베이스 로더는 웹페이지에 포함된 텍스트 콘텐츠를 직접 파싱하여 Document 객체로 변환하는 역할을 합니다.

이를 통해 뉴스 기사, 블로그 글, 공지사항과 같은 실시간으로 업데이트되는 웹 문서를 RAG 시스템의 지식 소스로 활용할 수 있습니다.

이번 실습에서는 실제 뉴스 기사를 예제로 사용하여, 웹페이지의 내용을 불러오고 텍스트 형태로 변환하는 과정을 살펴봅니다.

실습에 사용할 웹페이지는 다음과 같습니다.

<https://it.chosun.com/news/articleView.html?idxno=2023092111831>

```
In
from langchain_community.document_loaders import WebBaseLoader
```

실행 결과

```
WARNING:langchain_community.utils.user_agent:USER_AGENT environment variable not set, consider setting it to identify your requests.
```

```
In
loader = WebBaseLoader("https://it.chosun.com/news/articleView.html?idxno=2023092111831")
documents = loader.load()

#print(documents[0].page_content)
```

주석을 해제하고 코드를 실행하면, 해당 웹페이지에 포함된 본문 텍스트 전체를 불러와 확인할 수 있습니다.

웹페이지, PDF, CSV 등 서로 다른 형식의 문서들이 모두 텍스트 형태로 정상적으로 파싱된 것을 확인할 수 있습니다.

이제 이 텍스트를 **전처리(불필요한 요소 제거, 정제)** 한 뒤, Chunking과 Embedding 단계에 활용할 수 있습니다.

Step2 : TextSplitters 사용해보기

Text Splitter는 긴 텍스트 문서를 **의미를 유지한 작은 단위(Chunk)**로 분할하는 역할을 합니다.

LLM은 한 번에 처리할 수 있는 토큰 수에 제한이 있기 때문에, 문서를 그대로 입력하는 대신 Splitter를 통해 분할된 여러 Chunk를 입력받아 처리하게 됩니다.

이 과정을 통해 긴 문서에서도 토큰 길이 제약을 극복하고, 필요한 부분만 효율적으로 검색할 수 있습니다.

분할된 각 Chunk는 이후 단계에서 1:1로 Embedding되어 VectorStore에 저장되며, 이 Chunk 단위가 RAG 시스템에서 검색과 응답의 기본 단위가 됩니다.

In

```
from langchain_text_splitters import CharacterTextSplitter, RecursiveCharacterTextSplitter
```

CharacterTextSplitter는 하나의 고정된 구분자(separator)를 기준으로 텍스트를 분할하는 방식입니다. 구현이 단순하고 직관적이지만, 문서 구조에 따라 분할된 Chunk가 토큰 제한을 초과하는 경우가 발생할 수 있습니다.

반면, RecursiveCharacterTextSplitter는 줄바꿈, 문장 구분자, 구두점 등 여러 구분자를 순차적으로 적용하며 텍스트를 재귀적으로 분할합니다.

이 방식은 토큰 제한을 안정적으로 만족시키는 데 유리하지만, 분할 과정에서 의미적으로 완전하지 않은 문장 단위로 잘릴 수 있다는 단점이 있습니다.

단순한 구조의 문서나, 문단 구성이 명확한 텍스트의 경우에는 CharacterTextSplitter로도 충분합니다.

하지만 실제 서비스 환경에서는 문서 길이와 구조가 제각각인 경우가 많기 때문에, 대부분의 RAG 시스템에서는 RecursiveCharacterTextSplitter를 기본 선택지로 사용합니다.

이는 Chunk 크기를 안정적으로 제어하면서도 검색 실패를 줄이는 데 유리하기 때문입니다.

In

```
with open("/content/state_of_the_union.txt") as f:
    text = f.read()
```

In

```
#len은 어떤 기준으로 chunk size를 잘 것인가?의 기준이 되는 함수입니다.
#chunk_overlap은 chunk의 앞뒤로 다른 chunk와 설정한 size까지 겹칠 수 있도록 설정하는 것입니다.
text_splitter = CharacterTextSplitter(separator="\n\n", chunk_size=1000, chunk_overlap=100,
length_function = len,)
chunks = text_splitter.split_text(text)
```

Chunk의 내용을 확인해보겠습니다

In

```
print(chunks[0])
```

실행 결과

Madam Speaker, Madam Vice President, our First Lady and Second Gentleman. Members of Congress and the Cabinet. Justices of the Supreme Court. My fellow Americans.

Last year COVID-19 kept us apart. This year we are finally together again.

Tonight, we meet as Democrats Republicans and Independents. But most importantly as Americans.

With a duty to one another to the American people to the Constitution.

And with an unwavering resolve that freedom will always triumph over tyranny.

Six days ago, Russia's Vladimir Putin sought to shake the foundations of the free world thinking he could make it bend to his menacing ways. But he badly miscalculated.

He thought he could roll into Ukraine and the world would roll over. Instead he met a wall of strength he never imagined.

... (출력 일부 생략)

각 chunk의 길이를 확인해보겠습니다,

In

```
length = []
for chunk in chunks:
    length.append(len(chunk))

print(length)
```

실행 결과

[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966, 964, 977, 998, 948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]

토큰 단위로 텍스트 분할해보기

LLM은 문장을 단어가 아닌 토큰(token) 단위로 처리합니다. 따라서 사람이 인식하는 단어 길이나 문자 수는 실제 모델이 처리하는 입력 길이와 정확히 일치하지 않을 수 있습니다.

이로 인해 문자 수나 단어 수를 기준으로 텍스트를 분할할 경우, 모델의 입력 토큰 제한을 초과하거나 예상보다 훨씬 짧은 문맥만 전달되는 문제가 발생할 수 있습니다.

실제 서비스 환경에서는 이러한 문제를 방지하기 위해, 토큰 단위를 기준으로 텍스트를 분할하는 방식을 사용합니다. 이제 토큰 기준으로 텍스트를 분할해보겠습니다.

In

```
!pip install tiktoken
```

실행 결과

(패키지 설치 로그 생략)

```
In
import tiktoken
tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text):
    tokens = tokenizer.encode(
        text
    )
    return len(tokens)
```

```
In
tiktoken_length = []
for chunk in chunks:
    tiktoken_length.append(tiktoken_len(chunk))

print(length)
print(tiktoken_length)
```

실행 결과

```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888,
966, 964, 977, 998, 948, 925, 924, 989, 965, 938, 936, 981, 965, 771, 982, 972, 977, 984, 999, 968, 801]
[197, 198, 163, 190, 203, 182, 195, 197, 206, 205, 218, 148, 188, 205, 216, 215, 209, 224, 176, 187, 201,
197, 201, 215, 222, 202, 203, 204, 229, 206, 184, 204, 197, 194, 156, 200, 194, 221, 203, 225, 209, 187]
```

글자 수와 토큰 수의 차이를 확인할 수 있습니다 !

Step3 : TextEmbedding 사용해보기

Embedding은 텍스트를 컴퓨터가 계산할 수 있는 수치 벡터(vector) 형태로 변환하는 과정입니다. 이 벡터는 문장의 표면적인 형태가 아니라, 의미적 유사성을 반영하도록 설계되어 있습니다.

변환된 벡터는 VectorStore에 저장되거나, 새로운 질의(Query) 벡터와의 유사도 계산을 통해 의미적으로 가까운 문서를 검색하는 데 사용됩니다.

이러한 변환은 대규모 말뭉치로 사전 학습된 Embedding 전용 모델을 통해 이루어지며, RAG 시스템에서 Retrieval 성능을 결정하는 핵심 요소입니다.

이번 실습에서는 OpenAI 임베딩 모델을 사용해 텍스트를 벡터로 변환해보겠습니다.

```
In
import openai
```

genai 라이브러리의 list_models 함수를 사용하여 사용 가능한 모델들의 목록을 가져옵니다.

```
In
client = openai.OpenAI()
```

```
In
for model in client.models.list():
    if "embedding" in model.id:
        print(model.id)
```


실행 결과

```
text-embedding-3-small
text-embedding-3-large
text-embedding-ada-002
```

text-embedding-3-small은 가성비가 좋고, text-embedding-3-large는 성능이 더 강력합니다.

In

```
from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

만약 여러분들이 Gemini를 사용하여 구축중이시라면, embedding은 지역에 따라 사용이 제한됩니다.

주로 유럽권에서 제한되기 때문에, 다음 에러를 확인하신다면 Colab 파일의 서버 저장 위치를 확인 후, 다른 임베딩 모델로 변경해야 합니다.

Error embedding content: 400 User location is not supported for the API use.

In

```
#!curl ipinfo.io
```

실행 결과

(패키지 설치 로그 생략)

In

```
# 400 User location is not supported for the API use 오류가 발생한다면, 이 블록을 대신 실행해주세요

# ! pip install -q sentence_transformers

#from langchain.embeddings import HuggingFaceEmbeddings
#embedding_model = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

실행 결과

(패키지 설치 로그 생략)

embedding model 변수에 OpenAI 임베딩모델 혹은 huggingface의 임베딩모델이 할당되었을 것입니다.

embed_documents 멤버 함수를 사용하여 새 문장을 변환해보겠습니다

In

```
embeddings = embedding_model.embed_documents(
    [
        "This is red apple.",
        "This is yellow banana.",
        "This is green lime.",
    ]
)
```

임베딩으로 잘 변환되었는지 확인해보겠습니다

```
In
print(embeddings[1])
```

실행 결과

```
[0.006435394287109375, -0.0208587646484375, -0.0216064453125, 0.0235595703125, -0.0428466796875,
-0.004608154296875, 0.03973388671875, 0.05267333984375, -0.0018892288208007812, -0.0237579345703125,
0.0137939453125, 0.0159912109375, -0.044464111328125, -0.00013935565948486328, -0.007648468017578125,
0.02557373046875, 0.0254058837890625, 0.035491943359375, -0.0517578125, 0.01232147216796875,
0.0248260498046875, -0.00045800209045410156, 0.0193939208984375, 0.07525634765625, -0.0020503997802734375,
-0.00921630859375, 0.016815185546875, -0.007152557373046875, 0.02655029296875, -0.048492431640625,
0.043182373046875, -0.04254150390625, 0.01165008544921875, -0.032867431640625, -0.0333251953125,
-0.046142578125, 0.0019063949584960938, 0.0225677490234375, -0.01812744140625, 0.03253173828125,
0.0293121337890625, 0.011749267578125, -0.01442718505859375, 0.0030574798583984375, 0.0243377685546875,
0.048187255859375, -0.01355743408203125, 0.049346923828125, -0.040618896484375, 0.04400634765625,
0.04522705078125, 0.0195770263671875, 0.0203094482421875, 0.00183868408203125, 0.0183258056640625,
-0.0193939208984375, 0.030426025390625, 0.04461669921875, -0.030853271484375, -0.000415802001953125,
-0.00917816162109375, 0.005367279052734375, -0.06231689453125, 0.053009033203125, -0.001255035400390625,
-0.0166473388671875, -0.04693603515625, -0.0004734992980957031, 6.276369094848633e-05, 0.00685119628
... (출력 일부 생략)
```

```
In
len(embeddings[1])
```

실행 결과

1536

새로운 쿼리를 넣어, 임베딩끼리 유사도를 계산해보겠습니다

```
In
import numpy as np
from numpy import dot
from numpy.linalg import norm
def cos_sim(A, B):
    return dot(A, B)/(norm(A)*norm(B))
```

```
In
query = ["this is red fruit"]
```

```
In
e_query = embedding_model.embed_documents(query)
print(cos_sim(embeddings[0], e_query[0]))
print(cos_sim(embeddings[1], e_query[0]))
print(cos_sim(embeddings[2], e_query[0]))
```

실행 결과

```
0.7477942151308524
0.48995458116791124
0.4084112226829356
```

빨간 사과와 빨간 과일의 유사도가 많이 높게 나왔습니다!

임베딩 모델은 사용 언어나 필요에 따라 다양하게 교체하여 사용할 수 있습니다.
해당 링크에서 여러 목록을 확인하실 수 있습니다.
https://python.langchain.com/docs/integrations/text_embedding/

Step4 : VectorStore 사용해보기

VectorStore는 텍스트를 Embedding 모델을 통해 벡터(vector)로 변환한 뒤, 이를 저장하고 관리하는 저장소입니다. 이 저장소는 단순한 데이터 보관 공간이 아니라, 벡터 간의 유사도를 빠르게 계산하고 탐색하기 위한 인덱싱 구조를 함께 포함하고 있습니다.

문서나 쿼리가 Embedding된 이후에는, VectorStore를 통해 의미적으로 유사한 벡터를 효율적으로 검색할 수 있으며, 이 과정이 RAG 시스템의 Retrieval 단계를 담당하게 됩니다.

대표적인 VectorStore로는 Chroma, FAISS 등이 있으며, 각각 로컬 환경과 대규모 서비스 환경에서 널리 사용됩니다.

이번 실습에서는 구성이 단순하고 로컬 환경에서 바로 사용할 수 있는 ChromaDB를 사용해 VectorStore를 구성해보겠습니다.

```
In
!pip install chromadb
```

실행 결과

(패키지 설치 로그 생략)

```
In
!pip install langchain-chroma
```

실행 결과

(패키지 설치 로그 생략)

```
In
from langchain_chroma import Chroma
```

```
In
!pip install --upgrade opentelemetry-api
!pip install --upgrade opentelemetry-sdk
```

실행 결과

(패키지 설치 로그 생략)

제일 처음에 사용했던, PDF를 다시 사용하도록 합니다!

```
In
# 위에서 사용했던 코드입니다
loader = PyPDFLoader("/content/Demian.pdf")
pages = loader.load_and_split()

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0, length_function =
tiktoken_len)
docs = text_splitter.split_documents(pages)
```

```
In
#!pip show chromadb
```

실행 결과

(패키지 설치 로그 생략)

Chroma에 임베딩 시킵니다

```
In
db = Chroma.from_documents(docs, embedding_model)
```

이제 쿼리를 날려보겠습니다

```
In
query = "how Demian look like?"
docs = db.similarity_search(query)
```

```
In
print(docs[0].page_content)
```

실행 결과

```
DEMIAN
with a feeling of nausea, I noticed Demian's expression.
He had not thrust himself to the front but stood right
at the back, looking elegant and at ease as usual. His
glance seemed directed at the horse's head, and again it
. showed that deep, quiet, almost fanatical yet passionate
absorption. I could not help staring at him for some
moments and it was then that I felt aware of a very
uncanny sensation in my remote consciousness. I saw
Demian's face and remarked that it was not a boy's face
but a man's and then I saw, or rather became aware, that
it was not really the face of a man either; it had some
thing different about it, almost a feminine element. And
for the time being his face seemed neither masculine
... (출력 일부 생략)
```

Face, features, looks like 등 데미안의 생김새를 담고 있는 페이지가 출력되었습니다
굉장히 빠른 속도로 검색했습니다!

Step5 : Retriever 사용해보기

Retriever는 사용자의 질문을 Embedding 모델을 통해 벡터로 변환한 뒤, VectorStore에 저장된 문서 벡터들과 비교하여 의미적으로 가장 유사한 문서(Chunk)를 찾아 반환하는 역할을 합니다.

즉, Retriever는 RAG 시스템에서 “어떤 정보를 LLM에게 참고 자료로 줄 것인가”를 결정하는 핵심 컴포넌트이며, 검색 결과의 품질이 곧 최종 답변의 품질로 이어집니다.

```
In
!pip install -U langchain langchain-classic
```

(패키지 설치 로그 생략)

In

```
from langchain_classic.chains.retrieval_qa.base import RetrievalQA
```

긴 문서 전체를 한 번에 LLM에 전달하는 대신, Retriever와 LLM을 결합한 RetrievalQA 체인을 사용하여 문서에서 질문과 관련된 부분만 검색하고, 그 결과를 바탕으로 답변을 생성합니다.

이를 통해 길이가 긴 문서에서도 토큰 제한을 넘지 않으면서, 근거 기반의 질의응답을 수행할 수 있습니다.

In

```
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler

# OpenAI 모델로 변경
# streaming=True와 callbacks 설정을 통해 실시간 출력을 활성화합니다.
llm = ChatOpenAI(
    model="gpt-4o",          # 또는 "gpt-4o-mini"
    temperature=0.0,
    streaming=True,          # 실시간 출력을 켭니다
    callbacks=[StreamingStdOutCallbackHandler()], # 출력을 콘솔에 바로 뿌려줍니다
)
```

체인의 종류와 검색(Retrieval) 방식, 그리고 그에 따른 주요 파라미터를 설정합니다.

이 단계에서는 Retriever가 어떤 전략으로 문서를 검색할지, 그리고 몇 개의 문서를 LLM에게 전달할지를 결정하게 됩니다. 이 선택은 최종 답변의 품질과 직접적으로 연결됩니다.

예를 들어, MMR(Maximal Marginal Relevance) 방식은 쿼리와 유사도뿐만 아니라 문서 간의 중복을 줄이고 다양성을 확보하는 재정렬(Re-ranking) 전략입니다.

실무 환경에서는 단일 문서에 정보가 물리는 것을 방지하고, LLM이 보다 풍부한 문맥을 참고하도록 하기 위해 MMR 방식이 자주 사용됩니다.

In

```
qa = RetrievalQA.from_chain_type(llm, chain_type="stuff",
    retriever=db.as_retriever(
        search_type="mmr",
        search_kwargs={"k": 3, "fetch_k": 10}),
    return_source_documents=True)
```

위 코드에서 짚고 넘어갈 파라미터는 다음과 같습니다

- ◆ chain_type="stuff"

검색된 문서(Chunk)를 그대로 하나의 Prompt에 모두 삽입하는 방식입니다. 구조가 단순하고 이해하기 쉬워, RAG 구조를 처음 학습하거나 프로토타입을 만들 때 적합합니다. 단점으로는 문서 수가 많아질 경우 토큰 사용량이 빠르게 증가할 수 있습니다. 실무에서는 초기 검증 단계에서는 stuff를, 문서 수가 많아지면 map_reduce나 refine 방식으로 확장합니다.

- ◆ retriever

VectorStore에서 어떤 문서를 검색할지 결정하는 검색 모듈입니다. 검색 전략과 파라미터 설정에 따라 LLM이 참고하는 정보의 범위와 품질이 달라집니다.

- ◆ search_type="mmr"

MMR(Maximal Marginal Relevance) 검색 방식을 사용합니다. 쿼리와 유사도뿐만 아니라, 문서 간 중복을 줄여 다양한 문맥을 확보하는 Re-ranking 전략입니다. 실무 환경에서 단일 문서 편향을 줄이기 위해 자주 사용됩니다

- ♦ `search_kwargs={"k": 3, "fetch_k": 10}`

- `fetch_k`

VectorStore에서 우선적으로 가져올 후보 문서 개수입니다. Re-ranking 이전 단계에서 사용됩니다. - `k`

최종적으로 LLM에게 전달할 문서(Chunk)의 개수입니다.

일반적으로 `fetch_k > k` 로 설정하여 후보 풀을 넉넉히 확보한 뒤, 품질 좋은 문서만 선별하는 방식을 사용합니다.

- ♦ `return_source_documents=True`

답변 생성에 사용된 원문 문서(Chunk)를 함께 반환합니다. 이를 통해 답변의 출처를 사용자에게 표시하거나 검색 품질을 디버깅하고 RAG 성능을 평가할 수 있습니다. 실무 서비스에서는 거의 필수적으로 사용하는 옵션입니다.

```
In
query = "how demian looks like"
result = qa(query)
```

실행 결과

```
/tmp/ipykernel_12293/3336337621.py:2: LangChainDeprecationWarning: The method `Chain.__call__` was deprecated in langchain-classic 0.1.0 and will be removed in 1.0. Use `invoke` instead.
result = qa(query)
```

실행 결과

```
Demian is described as having a face that is neither distinctly masculine nor childish, neither old nor young, but rather timeless and bearing the mark of other periods of history. His face has an almost feminine element and is different from the rest of the people around him. It is suggested that his appearance is unique, almost like an animal, a spirit, or an image, making him unimaginably different from others.
```

마크다운 형식으로 출력해봅니다

```
In
from IPython.display import Markdown, display
display(Markdown(result["result"]))
```

RAG를 사용하지 않은 llm 호출도 시도해보세요!

```
In
llm2 = ChatOpenAI(
    model="gpt-4o")
request = llm2.invoke("how demian looks like")
display(Markdown(request.content))
```

Quiz

결과의 어떤 부분을 관찰하였을 때, RAG 시스템의 결과를 신뢰할 수 있겠다 생각하셨나요?

Answer

원문에서 답변의 출처를 확인할 수 있었습니다.

6. 완성 예제 — Demian RAG 파이프라인

앞의 Step 1~5를 하나의 **재현 가능한 파이프라인**으로 다시 조립한다. 단순 재실행이 아니라, 실무 RAG에서 품질을 좌우하는 다음 네 가지를 명시적으로 처리했다.

처리	왜 필요한가
OCR 잡음 정제	<code>Demian.pdf</code> 는 스캔 OCR 산출물이라 줄바꿈 분철(<code>invent-\ning</code>), 러닝헤더(<code>DEMIAN</code>), 워터마크가 섞여 있다. 정제하지 않으면 chunk 경계와 임베딩이 동시에 오염된다.
토큰 기준 chunking + overlap	문자 수 기준은 모델 입력 길이와 어긋난다. <code>tiktoken_len</code> 을 <code>length_function</code> 으로 넣고 overlap으로 문장이 잘리는 지점의 정보 손실을 보상한다.
Hybrid 검색 (MMR + BM25)	고유명사(Demian, Sinclair, Abraxas)는 dense embedding이 의외로 약하다. 어휘 정확 매칭(BM25)과 의미 검색을 앙상블한다.
Grounding 프롬프트 + 출처 표기	RAG의 신뢰는 "출처를 확인할 수 있다"에서 나온다(§ Quiz의 Answer). 근거가 없으면 답하지 않도록 강제하고, 페이지/chunk 번호를 인용시킨다.

설계 원칙 — 모든 하이퍼파라미터는 `CFG` 한 곳에서만 바꾼다. RAG는 변수가 많아 (chunk_size, overlap, k, fetch_k, λ, 임베딩 모델, 프롬프트) 통제하지 않으면 성능 변화의 원인을 사후에 귀속시킬 수 없다.

Step 0 · 라이브러리 설치

필요한 라이브러리를 모두 다운받습니다.

In

```
# — 코어 —
!pip install -U -q langchain langchain-core langchain-community langchain-classic
!pip install -U -q langchain-openai langchain-text-splitters

# — 로더 · 토큰라이저 · 벡터스토어 —
!pip install -U -q pypdf tiktoken chromadb langchain-chroma

# — 하이브리드 검색(BM25) · 평가 —
!pip install -U -q rank_bm25 nest_asyncio pandas
```

실행 결과

(패키지 설치 로그 생략)

Step 0 · 준비 (API 키 · 설정 · 토큰 카운터)

Text splitter 사용을 위한 준비입니다. `tiktoken_len` 은 앞 § Step2에서 정의한 것과 동일하며, 이후 splitter의 `length_function` 으로 주입되어 "문자 수"가 아닌 "토큰 수" 기준 분할을 만든다.

```

import os, re, warnings, textwrap
from typing import List
import numpy as np

warnings.filterwarnings("ignore")

# — API Key —————
try:
    from google.colab import userdata
    os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY")
except Exception:
    # 로컬/비-Colab 환경 대비
    assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 환경변수로 등록하세요."

# — 실험 설정: 모든 하이퍼파라미터는 여기서만 바꾼다 —————
CFG = {
    "PDF_PATH": "/content/Demian.pdf",
    "CHUNK_SIZE": 500, # tokens
    "CHUNK_OVERLAP": 80, # ≈16% - 문장이 잘린 지점의 정보 손실 보상
    "EMBED_MODEL": "text-embedding-3-small",
    "GEN_MODEL": "gpt-4o",
    "TEMPERATURE": 0.0, # 근거 기반 QA는 창의성이 아니라 재현성이 목표
    "K": 4, # LLM에 최종 투입할 chunk 수
    "FETCH_K": 20, # MMR 재순위 이전 후보 풀
    "LAMBDA_MULT": 0.5, # 1.0=유사도만, 0.0=다양성만
    "PERSIST_DIR": "/content/chroma_demian",
    "COLLECTION": "demian_v1",
}

# — 토큰 기준 길이 함수 —————
import tiktoken
tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text: str) -> int:
    return len(tokenizer.encode(text))

# — 모델 핸들 —————
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model=CFG["GEN_MODEL"], temperature=CFG["TEMPERATURE"])
embedding_model = OpenAIEmbeddings(model=CFG["EMBED_MODEL"])

print(f"준비 완료 · 생성={CFG['GEN_MODEL']} / 임베딩={CFG['EMBED_MODEL']}")
print(f"토큰 카운터 검증: tiktoken_len('Hello, Demian!') = {tiktoken_len('Hello, Demian!')}")

```

Step 1 Document loader

`PyPDFLoader` 는 PDF 1페이지 = Document 1개로 변환한다. 여기서는 `load_and_split()` 대신 `load()` 를 쓴다 — 기본 splitter가 곧바로 끼어들면 아래의 정제 단계를 적용할 기회를 잃기 때문이다. 분할은 Step 2에서 명시적으로 한다.

In

```

from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader(CFG["PDF_PATH"])
pages = loader.load()          # 페이지 단위 Document 리스트

print(f"로드된 페이지 수 : {len(pages)}")
print(f"메타데이터 예시 : {pages[10].metadata}")
print("-" * 70)
print(pages[10].page_content[:400])

```

Step 1-b · OCR 잡음 정제 (실무에서 가장 수익률이 높은 단계)

이 PDF는 **ScanFix(TM) Enhanced** + **Acrobat Paper Capture** 로 만들어진 **스캔 OCR 산출물**이다. 그대로 임베딩하면 다음 손실이 발생한다.

1. **줄바꿈 분철** — `invent-\ning`, `some-\nthing` 이 서로 다른 토큰열로 인코딩되어 검색 miss 유발
2. **러닝헤더/워터마크** — `DEMIAN`, `Downloaded from ...` 가 페이지마다 반복 → 임베딩 공간에서 모든 chunk가 서로 비슷해지는 *anisotropy* 를 키운다
3. **불규칙 공백/빈 페이지** — chunk 예산을 낭비

Advanced RAG 레퍼런스의 **인덱싱 최적화** 절이 지적하는 지점과 정확히 같다. 모델을 바꾸기 전에 **데이터를 먼저 고친다**.

In

```

# — 정제 규칙 —————
RUNNING_HEADER = re.compile(
    r"^s*(DEMIAN|TWO WOR\.?LDS|CAIN|THE THIEF|BEATRICE|EVA|"
    r"Downloaded from https?:/S+|[•\-\s]*|\d{1,3})s*$", re.M)

def clean(text: str) -> str:
    # (1) 줄바꿈 분철 복원: invent-\ning -> inventing
    text = re.sub(r"([A-Za-z])[-\u00ad\u2010]\s*\n\s*([a-z])", r"\1\2", text)
    text = text.replace("\u00ad", "")          # 잔여 soft hyphen
    # (2) 러닝헤더·페이지번호·워터마크 제거
    text = RUNNING_HEADER.sub("", text)
    # (3) 공백 정규화 (문단 경계 \n\n 은 splitter가 쓰므로 보존)
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()

before_chars = sum(len(p.page_content) for p in pages)

for p in pages:
    p.page_content = clean(p.page_content)

# 표지·백지·판권지처럼 실질 내용이 없는 페이지 제거
pages = [p for p in pages if tiktoken_len(p.page_content) > 30]

after_chars = sum(len(p.page_content) for p in pages)
print(f"정제 전 {before_chars:,}자 → 정제 후 {after_chars:,}자 "
      f"({100*(before_chars-after_chars)/before_chars:.1f}% 제거)")
print(f"유효 페이지 : {len(pages)}")
print("-" * 70)
print(pages[10].page_content[:400])

```

Step 2 Text splitters

`RecursiveCharacterTextSplitter` 를 사용한다. `CharacterTextSplitter` 는 단일 구분자만 쓰기 때문에 소설처럼 문단 길이가 들쭉날쭉한 문서에서 chunk가 토큰 제한을 넘거나 반대로 지나치게 짧아진다.

- `separators` 는 **거친 경계** → **고운 경계** 순으로 시도된다: 문단 → 줄 → 문장 → 단어 → 문자
- `length_function=tiktoken_len` : 문자 수가 아닌 **토큰 수**로 크기를 잴다
- `chunk_overlap=80` : 문장 중간에서 잘렸을 때 다음 chunk가 앞 문맥을 물고 시작하도록 한다. overlap이 0이면 경계에 걸친 서술(예: 데미안의 외모 묘사가 두 chunk에 나뉘는 경우)에서 recall이 떨어진다.

In

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", ". ", " ", ""],
    chunk_size=CFG["CHUNK_SIZE"],
    chunk_overlap=CFG["CHUNK_OVERLAP"],
    length_function=tiktoken_len,      # ← 토큰 기준
)

docs = text_splitter.split_documents(pages)

# 출처 표기·중복 제거·평가에 쓸 chunk_id 부여
for i, d in enumerate(docs):
    d.metadata["chunk_id"] = i

lens = [tiktoken_len(d.page_content) for d in docs]
print(f"chunk 개수      : {len(docs)}")
print(f"토큰 길이 분포 : min {min(lens)} / p25 {int(np.percentile(lens,25))} / "
      f"median {int(np.median(lens))} / p95 {int(np.percentile(lens,95))} / max {max(lens)}")
print(f"chunk_size 초과: {sum(l > CFG['CHUNK_SIZE'] for l in lens)} 개 ← 0이어야 정상")
print("-" * 70)
print(docs[40].page_content)
```

Step 3 Vector Empeddings

임베딩은 RAG 성능의 상한을 결정한다. 인덱싱에 들어가기 전에 "**이 임베딩이 우리 도메인에서 의미 구분을 하는가**" 를 값싼 sanity check로 먼저 확인한다. (전체 인덱싱은 API 비용과 시간이 들기 때문에, 검증은 항상 인덱싱 **앞**에 둔다.)

In

```

from numpy import dot
from numpy.linalg import norm

def cos_sim(A, B):
    return dot(A, B) / (norm(A) * norm(B))

# — sanity check : 실제 본문 chunk로 의미 구분이 되는지 확인 —————
probe_texts = [
    docs[40].page_content,          # 임의의 본문 chunk
    "Demian's face was neither a boy's nor a man's, almost timeless.", # 외모 묘사(가까워야)
    "The recipe requires two cups of flour and a pinch of salt.",      # 무관(멀어야)
]
E = embedding_model.embed_documents(probe_texts)
q = embedding_model.embed_query("What did Demian look like?")

print(f"임베딩 차원 : {len(E[0])}")
print(f"query ↔ 본문 chunk      : {cos_sim(E[0], q):.4f}")
print(f"query ↔ 외모 묘사 문장   : {cos_sim(E[1], q):.4f}   ← 가장 높아야 정상")
print(f"query ↔ 무관한 레시피 문장: {cos_sim(E[2], q):.4f}   ← 가장 낮아야 정상")

```

Step 3-b · VectorStore 구축 (Chroma)

`Chroma.from_documents()` 는 내부적으로 `embed_documents()` → 벡터 저장 → HNSW 인덱스 구성을 한 번에 수행한다. `persist_directory` 를 주면 세션이 끊겨도 재사용할 수 있어, 같은 인덱스 위에서 retriever·프롬프트만 바뀌가며 실험할 수 있다.

주의 — 같은 셀을 재실행하면 동일 문서가 **중복 적재**된다. 아래는 컬렉션을 먼저 비우고 다시 쌓는 방식(idempotent)으로 작성했다.

In

```

from langchain_chroma import Chroma
import chromadb

# 재실행 시 중복 적재 방지 - 같은 이름의 컬렉션이 있으면 삭제 후 재생성
try:
    _client = chromadb.PersistentClient(path=CFG["PERSIST_DIR"])
    _client.delete_collection(CFG["COLLECTION"])
    print(f"기존 컬렉션 '{CFG['COLLECTION']}' 삭제")
except Exception:
    pass

db = Chroma.from_documents(
    documents=docs,
    embedding=embedding_model,
    collection_name=CFG["COLLECTION"],
    persist_directory=CFG["PERSIST_DIR"],
)

try:
    n_vec = db._collection.count()
except Exception:
    n_vec = len(docs)
print(f"인덱싱 완료 : {n_vec:,} vectors (chunk {len(docs)}개와 일치해야 정상)")

```

Step 4 Retrievers

Retriever는 "LLM에게 무엇을 보여줄 것인가"를 결정한다. **검색이 틀리면 생성은 반드시 틀린다**. 세 가지를 만들어 비교한다.

Retriever	원리	강점 / 약점
Similarity	쿼리 벡터와 가장 가까운 top-k	단순·빠름 / 유사한 chunk가 몰려 문맥이 중복됨
MMR	<code>fetch_k</code> 개 후보를 뽑은 뒤 유사도 - λ · 기존 선택과의 중복으로 재순위	다양성 확보, 단일 페이지 편향 완화 / λ 튜닝 필요
Hybrid (BM25 + MMR)	어휘 정확매칭(sparse) + 의미검색(dense) 앙상블	고유명사·희귀어에 강함 / 인덱스 2벌 유지 비용

소설 텍스트는 `Abraxas`, `Sinclair`, `Kromer` 같은 **저빈도 고유명사**가 핵심 질의어가 되는 경우가 많다. Dense embedding 은 이런 토큰을 뭉개는 경향이 있어, BM25 앙상블의 효용이 특히 크다.

In

```
# — (1) Similarity vs (2) MMR —————
sim_retriever = db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": CFG["K"]},
)

mmr_retriever = db.as_retriever(
    search_type="mmr",
    search_kwargs={"k": CFG["K"],
                    "fetch_k": CFG["FETCH_K"],
                    "lambda_mult": CFG["LAMBDA_MULT"]},
)

probe = "How does Demian look like?"

def show(name, retriever, query):
    hits = retriever.invoke(query)
    pages_hit = [h.metadata.get("page") for h in hits]
    print(f"[{name}] 회수 {len(hits)}개 · 페이지 {pages_hit} · "
          f"고유 페이지 {len(set(pages_hit))}개")
    for h in hits:
        head = h.page_content.replace("\n", " ")[:88]
        pg = str(h.metadata.get("page"))
        print(f"    L p.{pg:>3} #{h.metadata['chunk_id']:>4} | {head}...")
    print()

show("similarity", sim_retriever, probe)
show("mmr", mmr_retriever, probe)
```

In

```
# — (3) Hybrid : BM25(sparse) + MMR(dense) 앙상블 —————
from langchain_community.retrievers import BM25Retriever
try:
    from langchain_classic.retrievers import EnsembleRetriever
except ImportError:
    # 구버전 호환
    from langchain.retrievers import EnsembleRetriever

bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = CFG["K"]

hybrid_retriever = EnsembleRetriever(
    retrievers=[bm25_retriever, mmr_retriever],
    weights=[0.4, 0.6],      # 어휘 매칭 0.4 : 의미 매칭 0.6
)

# 고유명사 질의에서 세 retriever의 차이를 확인
for q in ["Who is Abraxas?", "Franz Kromer가 싱클레어를 헐박한 이유는?"]:
    print("=" * 78)
    print("QUERY:", q)
    print("=" * 78)
    show("mmr", mmr_retriever, q)
    show("hybrid", hybrid_retriever, q)
```

Step 5 Question Answering

마지막 단계. 여기서 두 가지를 명시적으로 설계한다.

① **Grounding 프롬프트** — RAG의 실패는 보통 "검색은 됐는데 모델이 자기 사전지식으로 덧칠"할 때 생긴다. 『데미안』은 학습 코퍼스에 이미 들어 있을 가능성이 매우 높은 정전(canon)이라, 이 위험이 특히 크다. 따라서 *context 밖의 지식 사용 금지*와 *근거 부족 시 거절*을 시스템 프롬프트로 강제한다.

② **출처 표기** — § Quiz의 답("원문에서 답변의 출처를 확인할 수 있었다")을 시스템 차원에서 보장한다. [p.페이지 #chunk_id] 형식으로 인용시키면 사람이 즉시 원문 대조를 할 수 있고, 이것이 곧 다음 절 평가에서 **faithfulness** 측정의 기반이 된다.

체인은 LCEL(LangChain Expression Language)로 구성한다. **RetrievalQA** (§ Step5 앞부분에서 사용)는 **langchain-classic**으로 이관된 레거시 API이며, LCEL은 각 단계를 조립·교체·관찰할 수 있어 Modular RAG 관점에 부합한다.

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

```

```
SYSTEM = """당신은 헤르만 헤세의 소설 『데미안』 원문만을 근거로 답하는 문학 연구 조교입니다.
```

```
반드시 지킬 규칙:
```

1. 아래 <context> 안의 내용만 근거로 삼습니다. 당신이 이미 알고 있는 『데미안』 지식으로 보충하거나 추론으로 메우지 마십시오.
2. context에 근거가 없으면 "제공된 원문에서는 확인할 수 없습니다"라고 답하고, 무엇이 부족한지 한 문장으로 덧붙입니다. 추측하지 마십시오.
3. 각 주장 끝에 근거 출처를 [p.{페이지}] #{chunk_id}] 형식으로 표기합니다.
4. 한국어로, 3~6문장으로 답합니다.
5. 원문 인용이 필요하다면 한 문장 이내로 짧게 인용하고 즉시 출처를 답니다."""

```

USER = """<context>
{context}
</context>

```

```
질문: {question}"""
```

```
prompt = ChatPromptTemplate.from_messages([("system", SYSTEM), ("human", USER)])
```

```

def format_docs(docs_) -> str:
    """검색된 chunk를 출처 헤더와 함께 직렬화한다.
    모델이 인용할 수 있도록 메타데이터를 본문과 같은 평문 안에 넣는 것이 핵심."""
    return "\n\n--\n\n".join(
        f"[p.{d.metadata.get('page')}] #{d.metadata.get('chunk_id')}] \n{d.page_content}"
        for d in docs_
    )

```

```
# retriever → context 직렬화 → 프롬프트 → LLM → 문자열
```

```

rag_chain = (
    RunnableParallel(
        context=hybrid_retriever | format_docs,
        question=RunnablePassthrough(),
    )
    | prompt
    | llm
    | StrOutputParser()
)

```

```
# 출처를 함께 반환하는 래퍼 (return_source_documents=True 와 동일한 목적)
```

```

def ask(question: str, retriever=hybrid_retriever, verbose: bool = True):
    sources = retriever.invoke(question)
    answer = (prompt | llm | StrOutputParser()).invoke(
        {"context": format_docs(sources), "question": question})
    if verbose:
        print("Q:", question)
        print("-" * 78)
        print(textwrap.fill(answer, 78))
        print("-" * 78)
        print("근거 chunk:")
        for s in sources:
            head = s.page_content.replace("\n", " ")[:70]
            pg = str(s.metadata.get("page"))
            print(f"    • p.{pg}>3} #{s.metadata['chunk_id']>4} | {head}...")
        print()
    return {"answer": answer, "sources": sources}

```

```
_ = ask("데미안의 외모는 어떻게 묘사되는가?")
```

Step 5-b · 검증 ① RAG vs No-RAG, ② 환각 저항성

RAG를 붙였다는 사실만으로는 아무것도 증명되지 않는다. 두 가지를 확인한다.

- **대조군** — 같은 질문을 RAG 없이 던졌을 때와 비교한다. RAG 답변은 *원문 근거를 지목*하고, No-RAG 답변은 *일반론으로 흐른*다는 차이가 관찰되어야 한다.
- **환각 저항성(negative probe)** — 원문에 존재하지 않는 사실을 물었을 때 **"확인할 수 없습니다"**라고 **거절**하는지를 본다. 거절하지 못하면 그 시스템의 모든 긍정 답변도 신뢰할 수 없다. 이것이 실무에서 가장 먼저 봐야 할 지표다.

In

```
# — ① 대조군: RAG 미사용 —————
llm_plain = ChatOpenAI(model=CFG["GEN_MODEL"], temperature=0.0)

q = "데미안의 외모는 어떻게 묘사되는가?"
print("┌" + "=" * 76 + "┐")
print("┆ [No-RAG] 모델 내부 지식만 사용" + " " * 44 + "┆")
print("└" + "=" * 76 + "┘")
print(textwrap.fill(llm_plain.invoke(q).content, 78))
print()

# — ② 환각 저항성: 원문에 없는 사실 —————
negative_probes = [
    "데미안이 졸업한 대학의 정확한 이름과 졸업 연도는?",
    "싱클레어의 여동생이 결혼한 상대의 직업은 무엇인가?",
]
print("┌" + "=" * 76 + "┐")
print("┆ [Negative probe] 원문에 없는 정보 - 거절해야 정상" + " " * 26 + "┆")
print("└" + "=" * 76 + "┘")
for p in negative_probes:
    _ = ask(p)
```

Step 5-c · 배치 질의 (파이프라인 최종 확인)

난이도가 다른 5개 질문으로 전체 파이프라인을 한 번에 통과시킨다.

1. **사실 회수형** — 단일 chunk에 답이 있음
2. **관계 추론형** — 여러 chunk를 종합해야 함
3. **상징 해석형** — 소설의 핵심 모티프(Abraxas, 알을 깨는 새)
4. **한국어 질의 · 영어 원문** — cross-lingual retrieval 검증
5. **부재 정보** — 거절해야 함

In

```

QUESTIONS = [
    "Franz Kromer는 싱클레어에게 무엇을 요구했는가?",           # 1. 사실 회수
    "데미안이 카인(Cain)의 이야기를 새롭게 해석한 방식은 무엇인가?", # 2. 관계 추론
    "Abraxas는 이 소설에서 어떤 의미를 지니는가?",               # 3. 상징 해석
    "싱클레어가 말하는 '두 세계(two worlds)'란 무엇인가?",       # 4. cross-lingual
    "이 소설에서 데미안이 사망한 정확한 날짜는 언제인가?",      # 5. 부재 정보
]

results = []
for i, q in enumerate(QUESTIONS, 1):
    print(f"\n{'-'*78}\n[{i}/{len(QUESTIONS)}]")
    results.append(ask(q))

```

7. (확장) 정량 평가 — RAGAS

레퍼런스 노트북 *Rag_evaluation*의 프레임을 이 파이프라인에 그대로 적용한다. RAG 평가는 **검색**과 **생성**을 분리해서 봐야 원인을 귀속시킬 수 있다.

지표	무엇을 재는가	낮을 때 손대야 할 곳
context_precision	회수된 chunk 중 실제로 쓸모 있는 비율	k 축소, MMR λ , 재순위(reranker)
context_recall	정답에 필요한 정보를 빠짐없이 회수했는가	chunk_size / overlap , fetch_k , hybrid 가중치
faithfulness	답변이 context에서만 나왔는가 (= 환각의 역수)	프롬프트 grounding, temperature
answer_relevancy	답변이 질문에 실제로 답했는가	프롬프트, 생성 모델

핵심 진단 규칙: **recall**이 낮으면 **chunking/검색 문제**, **faithfulness**가 낮으면 **프롬프트/생성 문제**. 이 둘을 뭉뚱그리면 엉뚱한 곳을 튜닝하게 된다.

In

```

!pip install -U -q ragas datasets
import nest_asyncio; nest_asyncio.apply() # Colab 이벤트 루프 충돌 방지

```



```

from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall, context_precision

# 평가셋: 원문을 직접 읽고 만든 ground truth (부재 정보 문항 포함)
eval_questions = [
    "Franz Kromer는 싱클레어에게 무엇을 요구했는가?",
    "싱클레어가 말하는 '두 세계(two worlds)'란 무엇인가?",
    "이 소설에서 데미안이 사망한 정확한 날짜는 언제인가?",
]
eval_ground_truths = [
    "크로머는 싱클레어가 지어낸 사과 도둑질 이야기를 빌미로 그를 협박하며 돈을 요구했고, "
    "싱클레어는 그 요구에 시달리며 죄책감과 공포 속에 빠져든다.",
    "밝고 질서 있는 부모의 세계(경건함·규율·안전)와, 그 바깥의 어둡고 금지된 세계"
    "(하인·범죄·성·공포)라는 두 개의 대립된 영역을 뜻한다.",
    "원문에는 데미안의 사망 날짜가 제시되지 않는다.",
]

records = {"user_input": [], "response": [], "retrieved_contexts": [], "reference": []}
for q, gt in zip(eval_questions, eval_ground_truths):
    r = ask(q, verbose=False)
    records["user_input"].append(q)
    records["response"].append(r["answer"])
    records["retrieved_contexts"].append([d.page_content for d in r["sources"]])
    records["reference"].append(gt)

dataset = Dataset.from_dict(records)

# 판사(judge) 모델은 생성 모델과 분리 - 자기 답을 자기가 채점하는 편향을 줄인다
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

evaluator_llm = LangchainLLMWrapper(ChatOpenAI(model="gpt-4o-mini", temperature=0))
evaluator_embeddings = LangchainEmbeddingsWrapper(OpenAIEmbeddings(model="text-embedding-3-small"))

result = evaluate(
    dataset=dataset,
    metrics=[context_precision, context_recall, faithfulness, answer_relevancy],
    llm=evaluator_llm,
    embeddings=evaluator_embeddings,
)

print(result)
result.to_pandas()

```

결과 해석 가이드

- **context_precision** ↓ — **k**가 과하거나 chunk가 너무 길다. **k**를 3으로 줄이고 **λ**를 0.7로 올려본다.
- **context_recall** ↓ — 정답이 chunk 경계에서 잘렸을 가능성. **chunk_overlap**을 120으로, **fetch_k**를 30으로 올려 재 측정한다.
- **faithfulness** ↓ — 모델이 『데미안』 사전지식으로 덧칠한 것. temperature 확인 후 시스템 프롬프트의 규칙 1·2를 더 강하게 명시한다.
- **부재 정보 문항의 faithfulness**가 특히 중요하다. 거절하지 못하고 그럴듯한 날짜를 지어내면, 이 시스템은 어떤 긍정 답변도 신뢰할 수 없다.

한 번에 여러 하이퍼파라미터를 바꾸지 말 것. RAG 튜닝은 변수가 많아 **한 번에 하나씩(one-factor-at-a-time)** 바꿔야 성능 변화의 원인을 귀속시킬 수 있다.

8. 정리 · 한계 · 다음 단계

무엇을 만들었나

```
Demian.pdf (스캔 OCR)
  | PyPDFLoader.load()          ← 페이지 = Document
  | clean()                     ← 분철 복원 · 러닝헤더 제거 · 공백 정규화    [A]
  ▼
RecursiveCharacterTextSplitter(500 tok / overlap 80, length=tiktoken_len) [B]
  ▼
OpenAIEmbeddings(text-embedding-3-small, 1536d) [C]
  ▼
Chroma (persist, HNSW) [D]
  ▼
Hybrid Retriever = BM25(0.4) ⊗ MMR(0.6, k=4, fetch_k=20, λ=0.5) [1~3]
  ▼
Grounding Prompt (context-only · 근거 부족 시 거절 · [p.# ] 인용) [4]
  ▼
gpt-4o (T=0) → 답변 + 출처 [5]
  ▼
RAGAS (precision / recall / faithfulness / relevancy)
```

[A]~[D], [1]~[5] 는 첨부 다이어그램의 Data Preparation 단계와 Retrieval 단계 번호에 대응한다.

알게 된 것

1. **정제가 모델 교체보다 싸고 효과적이다.** 스캔 OCR 문서에서 분철 복원과 러닝헤더 제거만으로 검색 품질이 눈에 띄게 달라진다. 임베딩 모델을 **-large** 로 올리는 것은 그 다음 순서다.
2. **토큰 기준 chunking은 선택이 아니라 전제다.** 문자 수 기준은 모델의 실제 입력 예산과 체계적으로 어긋나며, 그 오차가 문서마다 달라 통제가 불가능하다.
3. **고유명사 검색에서 dense는 생각보다 약하다.** **Abraxas**, **Kromer** 같은 저빈도 토큰은 BM25 양상불이 실질적 개선을 준다 — Advanced RAG의 "hybrid search"가 권장되는 실제 이유.
4. **환각 저항성이 최우선 지표다.** 거절해야 할 질문에 거절하지 못하는 시스템은, 맞은 답변조차 우연인지 실력인지 구분할 수 없다.

한계

- **평가셋이 3문항**으로 통계적 신뢰구간이 없다. 최소 30~50문항, 난이도 계층별 층화가 필요하다.
- **Chunk 경계가 여전히 서사 단위와 어긋난다.** 소설은 장(chapter)·장면 단위 구조를 갖는데 고정 길이 분할은 이를 무시한다. → semantic chunking 또는 장 단위 parent-child 인덱싱.
- **Retriever 가중치 [0.4, 0.6] 이 근거 없이 정해졌다.** 평가셋을 키운 뒤 grid search로 확정해야 한다.
- **비용 관측이 없다.** 실무에서는 답변 1건당 토큰/지연/비용을 함께 로깅해야 품질-비용 곡선을 그릴 수 있다.

다음 단계 (레퍼런스 노트북 연계)

방향	구체 기법	기대 효과
인덱싱 최적화	Semantic chunking, Parent-Document Retriever	chunk 경계 손실 제거
쿼리 최적화	Multi-Query, HyDE, Query Decomposition	모호한 질의 recall ↑
증강	Cross-encoder Reranker (예: bge-reranker)	precision ↑, k 축소 가능
구조	Self-RAG / CRAG (검색 필요 여부·품질을 모델이 판단)	불필요 검색 제거, 환각 ↓
평가	평가셋 50문항 + LLM-as-judge 이중 채점	튜닝 결과의 유의성 확보

완성 제출 · 추병곤 · Demian RAG Pipeline (LangChain + Chroma + Hybrid Retrieval + RAGAS)